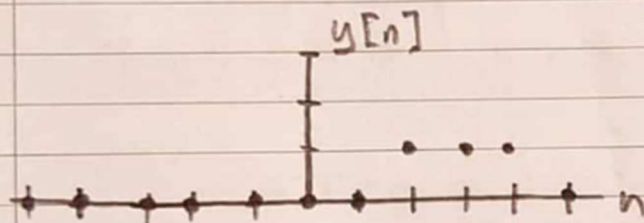
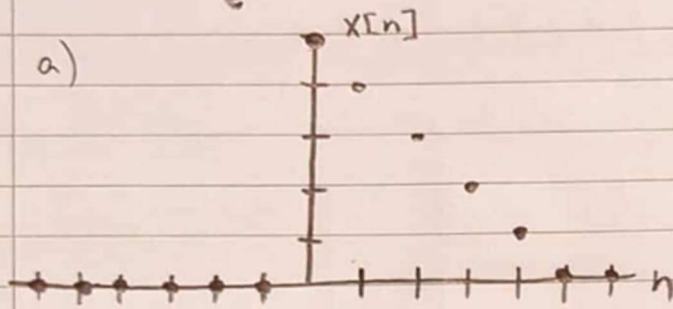


# Problem set 1

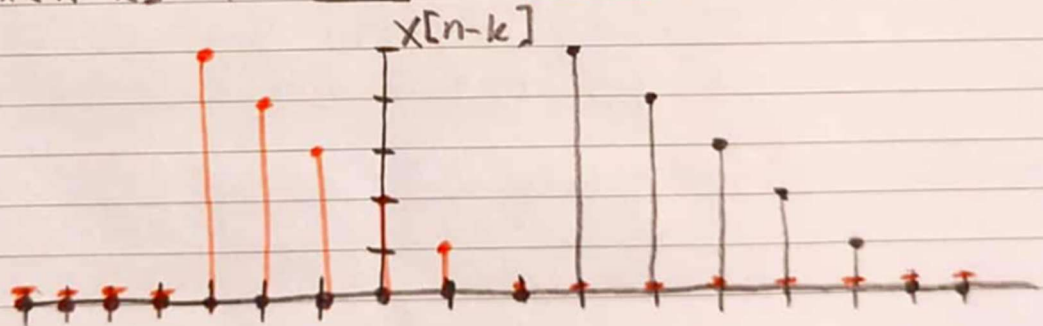
## Problem 1

$$x[n] = \begin{cases} 5-n & 0 \leq n \leq 4 \\ 0 & \text{otherwise} \end{cases} \quad y[n] = \begin{cases} 1 & 2 \leq n \leq 4 \\ 0 & \text{otherwise} \end{cases}$$

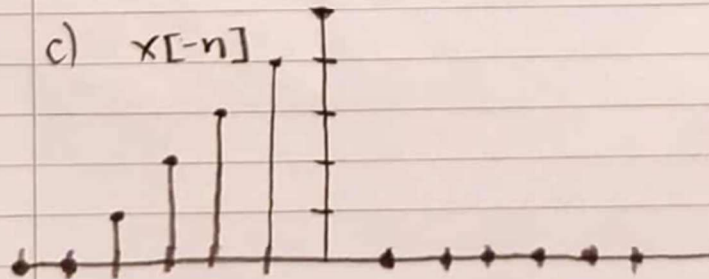
a)



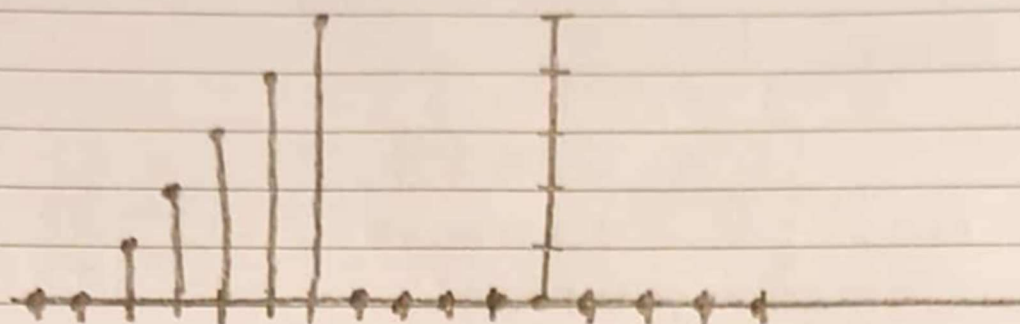
b)  $x[n-k]$  for  $k=3$  and  $k=-3$



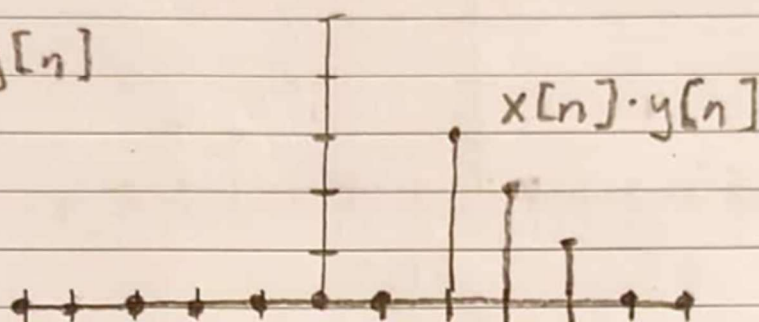
c)  $x[-n]$



d)  $x[5-n]$



e)  $x[n] \cdot y[n]$



$$f) x[n] = \sum_{k=-\infty}^{\infty} x[k] \delta[n-k]$$

$$= x[0] \delta[n] + x[1] \delta[n-1] + x[2] \delta[n-2] + x[3] \delta[n-3] + x[4] \delta[n-4]$$

$$g) y[n] = u[n-2] - u[n-5]$$

$$h) E = \sum_{n=-\infty}^{\infty} |x[n]|^2 = 5^2 + 4^2 + 3^2 + 2^2 + 1^2 = 55$$

## Problem 2

a)  $F_s = 6000 \text{ Hz}$

$$f_1 = \frac{F_1}{F_s} \pmod{1}$$

$$\cos(2\pi(f_1 + k)n) = \cos(2\pi f_1 n) \quad k \in \mathbb{Z}$$

$$\frac{F_1}{F_s} = f_1 + k, \quad k \in \mathbb{Z}$$

$$F_1 = f_1 F_s + k F_s$$

$$F_1 = f_1 \cdot 6000 \text{ Hz} + k \cdot 6000 \text{ Hz}, \quad k \in \mathbb{Z}$$

You can add any multiple of  $F_s$  to  $f_1 F_s$  and still get the same sampled sequence when you sample  $F_1$ .

2b)

```
import numpy as np
import matplotlib.pyplot as plt

def harmonic_sequence(A, f_1, n_array):
    return A*np.cos(2*np.pi*f_1*n_array)

#F_S = 6000Hz -> 24k samples in 4 sec
F_S = 6000
F_1 = 2000
f_1 = F_1/F_S
n_values = np.arange(24000)
x_n = harmonic_sequence(1, f_1, n_values)
```

2c)

```
import numpy as np
import matplotlib.pyplot as plt
import sounddevice as sd

def harmonic_sequence(A, f_1, n_array):
    |   return A*np.cos(2*np.pi*f_1*n_array)

F_S1, F_S2, F_S3 = 1000, 3000, 12000
f_1 = 0.3
#Number of samples in 4 sec
N_1, N_2, N_3 = 4*F_S1, 4*F_S2, 4*F_S3

n_1_values = np.arange(N_1)
n_2_values = np.arange(N_2)
n_3_values = np.arange(N_3)

x_n1 = harmonic_sequence(1,f_1, n_1_values)
x_n2 = harmonic_sequence(1,f_1, n_2_values)
x_n3 =harmonic_sequence(1,f_1, n_3_values)

#Pitch increases because F_1 increases
sd.play(x_n1, samplerate=F_S1)
sd.wait()
sd.play(x_n2, samplerate=F_S2)
sd.wait()
sd.play(x_n3, samplerate=F_S3)
sd.wait()
```

2d)

```
import numpy as np
import matplotlib.pyplot as plt
import sounddevice as sd

def harmonic_sequence(A, f_1, n_array):
    return A*np.cos(2*np.pi*f_1*n_array)

F_S = 8000
F1_1000, F1_3000, F1_6000 = 1000, 3000, 6000

#Number of samples in 4 sec
N = 4*F_S
f1_0, f1_1, f1_2 = F1_1000/F_S, F1_3000/F_S, F1_6000/F_S

n_values = np.arange(N)

x_n1 = harmonic_sequence(1, f1_0, n_values)
x_n2 = harmonic_sequence(1, f1_1, n_values)
x_n3 = harmonic_sequence(1, f1_2, n_values)

#2000Hz note, f1 = 0.125 (f1 < 0.5)
sd.play(x_n1, samplerate=F_S)
sd.wait()
#3000Hz note f1 = 0.375 (f1 < 0.5)
sd.play(x_n2, samplerate=F_S)
sd.wait()
#2000Hz note because of aliasing (F1 was 6000Hz). f1 = 0.75 and f > 0.5
sd.play(x_n3, samplerate=F_S)
sd.wait()
```

Problem 3

a)  $y[n] = x[n] - x^2[n-1]$

Non-linear: quadratic term, TI, Causal

b)  $y[n] = n x[n] + 2x[n-2]$

Time-variant: factor  $n$ , Linear, causal

c)  $y[n] = x[n] - x[n-1]$

All apply. Time shifting is linear

d)  $y[n] = x[n] + 3x[n+4]$

Non-causal because the series depends on future inputs. Also TI and linear



# Problem 4

$$a) I. y[n] = x[n] + 2x[n-1] + x[n-2]$$

$$II. y[n] = -0,9y[n-1] + x[n]$$

$$x[n] = \delta[n]$$

$$h_1[n] = \delta[n] + 2\delta[n-1] + \delta[n-2]$$

$h_2$ : Assume  $h[n] = 0$  for  $n < 0$

$$h_2[n] = (-0,9)^n u(n)$$

b)  $h_1[n]$  has a finite impulse response (FIR) because only 3 values are non-zero

$h_2[n]$  is an infinite impulse response (IIR) because it never dies out, it only approaches 0

c) BIBO stable if

$$|x[n]| \leq M_x < \infty \Rightarrow |y[n]| \leq M_y < \infty \quad \forall n$$

~~Both systems are BIBO stable because the output is bounded for both~~

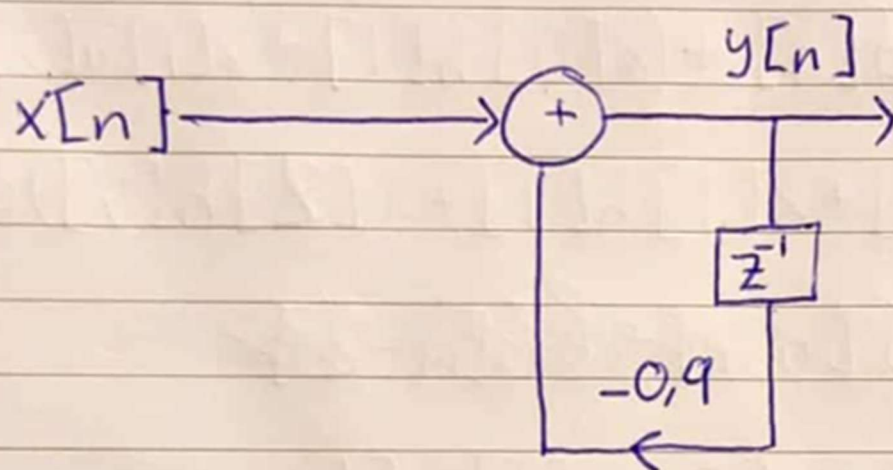
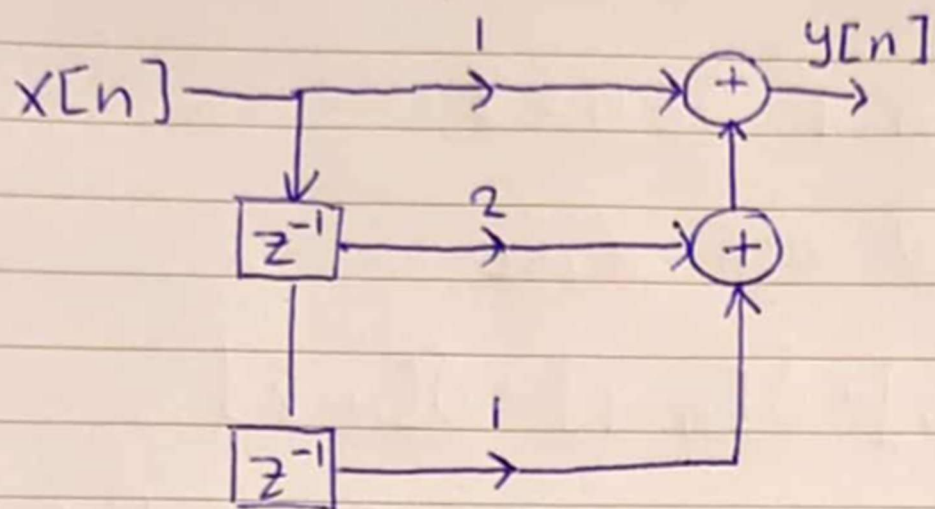
~~I is not BIBO stable because it can diverge. Example:  $x[n] = u[n]$ . The output will continue to grow.~~

I is stable because the output is the sum of 3 bounded inputs

II is stable because  $h_2[n]$  is absolute summable. This applies to LTI systems.

$$\sum_{n=-\infty}^{\infty} |(-0,9)^n| = \frac{1}{1-0,9} = 10$$

4d)



# Problem 5

$$x[n] = \begin{cases} n+1 & 0 \leq n \leq 2 \\ 0 & \text{otherwise} \end{cases}$$

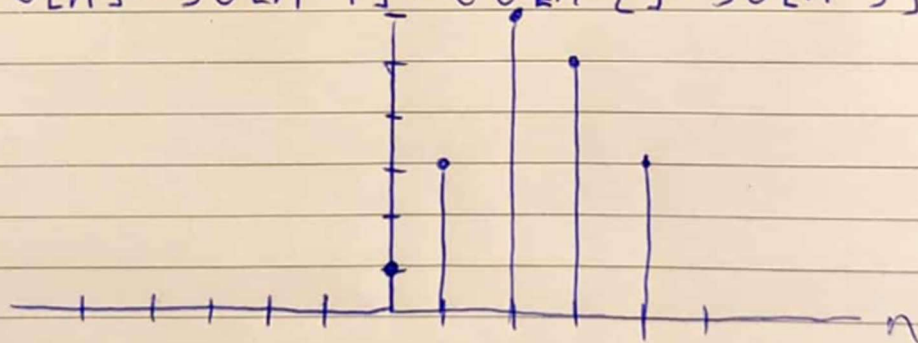
$$h_1[n] = \delta[n] + \delta[n-1] + \delta[n-2]$$

$$\begin{aligned} a) y_1[n] &= x[n] * h_1[n] = \sum_k x[k] h_1[n-k] \\ &= x[0] h_1[n] + (x[0] h_1[n-1] + x[1] h_1[n-2]) \\ &\quad + (x[0] h_1[n-2] + x[1] h_1[n-3] + x[2] h_1[n-4]) \end{aligned}$$

~~$$+ \delta[n] + 3\delta[n-1] + 6\delta[n-2]$$~~

$$\begin{aligned} &+ (x[1] h_1[n-2] + x[2] h_1[n-3]) \delta[n-3] \\ &+ x[2] h_1[n-4] \delta[n-4] \end{aligned}$$

$$= \delta[n] + 3\delta[n-1] + 6\delta[n-2] + 5\delta[n-3] + 3\delta[n-4]$$





5b)

```
import numpy as np
import matplotlib.pyplot as plt

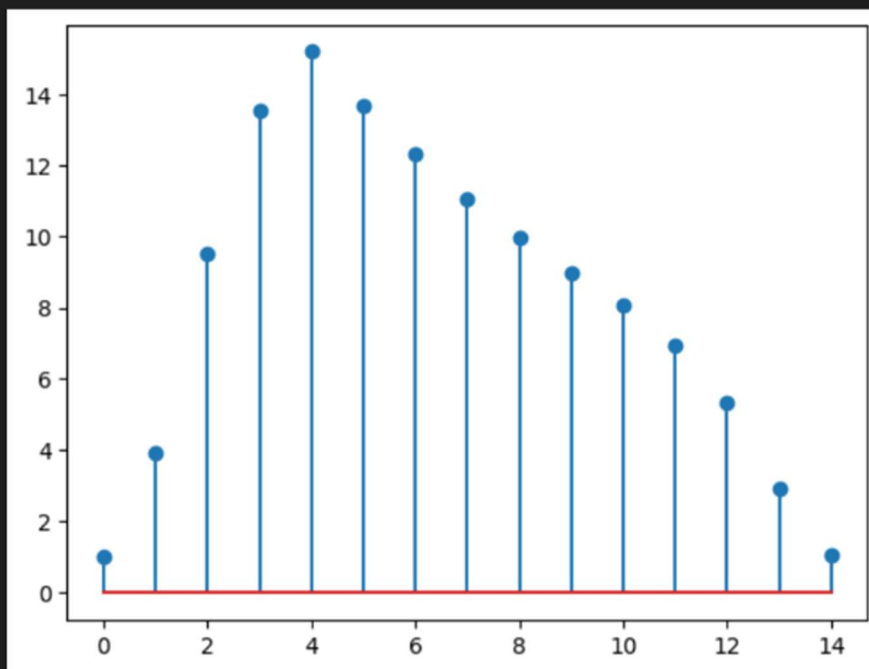
x_n = np.array([1,2,3])
h1_n = np.array([1,1,1])
h2_n = np.array([0.9**n for n in range(11)])

y1_n = np.convolve(x_n, h1_n)
y2_n = np.convolve(y1_n, h2_n)

plt.stem(y2_n)
```

✓ 0.1s

<StemContainer object of 3 artists>



c) The convolution of 2 sequences  
has length  $L_{y_1} = L_x + L_{h_1} - 1 = 5$   
 $L_{y_2} = L_{y_1} + L_{h_2} - 1 = 5 + 11 - 1 = 15$

5d)

$y1[n]$  looks different, but  $y2[n]$  remains the same.

This is because convolution is commutative and changing the order of  $h1$  and  $h2$  doesn't change the result

```
import numpy as np
import matplotlib.pyplot as plt

x_n = np.array([1,2,3])
h1_n = np.array([1,1,1])
h2_n = np.array([0.9**n for n in range(11)])

y1_n = np.convolve(x_n, h2_n)
y2_n = np.convolve(y1_n, h1_n)

plt.stem(y1_n)
plt.stem(y2_n)
```

✓ 0.1s

<StemContainer object of 3 artists>

